Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte

Rapport de Semaine 7

Observabilité, Performance & Déploiement Final

Étudiants :
Ahcène Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
7 / 7

23 avril 2026

Table des matières

1 Observabilité Complète **2**
1.1 Traçage Distribué avec Jaeger 2
1.2 Collecte de Métriques avec Prometheus 2
1.3 Dashboards Grafana Personnalisés 2

2 Interface de pilotage des Alertes **2**

3 Tests de Charge et Performance (K6) **3**
3.1 Scénarios de Tests 3
3.2 Résultats et Optimisations 3

4 Déploiement Final avec Helm **3**
4.1 Organisation des Charts 3
4.2 Configuration Ingress et Sécurité 3

5 Documentation et Finalisation **4**
5.1 Documentation Technique 4
5.2 Préparation de la Démonstration 4

6 Livrables et État Final **4**

1. Observabilité Complète

L'architecture distribuée de la plateforme a été enrichie d'une stack d'observabilité complète permettant de surveiller la santé du système et de diagnostiquer les problèmes en temps réel.

1.1. Traçage Distribué avec Jaeger

Chaque microservice (Java et Node.js) intègre l'**agent OpenTelemetry** pour collecter et exporter des traces distribuées. Cela permet de visualiser le cheminement complet d'une requête, de la Gateway GraphQL jusqu'aux services backend et aux bases de données.

— **Instrumentation automatique** : Utilisation du `opentelemetry-javaagent.jar` pour les services Java et du SDK OpenTelemetry pour Node.js.

— **Propagation de contexte** : Les en-têtes de trace sont transmis lors des appels gRPC et des messages Kafka.

1.2. Collecte de Métriques avec Prometheus

Un serveur **Prometheus** centralise les métriques techniques et métiers du cluster.

— **Spring Boot Actuator** : Expose les métriques JVM, HTTP et Kafka via le endpoint `/actuator/prometheus`.

— **NestJS Metrics** : Intégration de métriques Prometheus personnalisées pour le service de localisation.

— **Kafka Exporter** : Surveillance du lag des consommateurs et du débit des topics.

1.3. Dashboards Grafana Personnalisés

Nous avons conçu des tableaux de bord **Grafana** pour offrir une visibilité multi-niveaux :

1. **System Health** : CPU, Mémoire, Taux d'erreurs par service.
2. **Business Metrics** : Nombre de véhicules en mouvement, alertes actives par sévérité, taux de disponibilité de la flotte.
3. **Kafka Traffic** : Visualisation des flux de messages entre les services.

2. Interface de pilotage des Alertes

Une nouvelle page `/alerts` a été intégrée au frontend pour permettre aux opérateurs de visualiser et d'acquitter les alertes en temps réel.

> **Choix d'architecture :** Contrairement aux autres modules métiers, la gestion des alertes a été implémentée directement au sein de l'application **Shell** et non comme un micro-frontend (MFE) séparé. Ce choix permet d'éviter la complexité

> disproportionnée qu'induirait la création d'un dépôt et d'un pipeline de déploiement dédié pour un composant aussi transverse et léger.

Cette interface consomme le flux de messages du `events-service` via GraphQL et permet un pilotage réactif de la flotte directement depuis le cockpit central.

3. Tests de Charge et Performance (K6)

Pour valider la robustesse de l'architecture face à une montée en charge, nous avons utilisé **K6**.

3.1. Scénarios de Tests

Plusieurs scénarios ont été modélisés dans le dossier `load-tests/` :

— **Flux GPS Massif** : Simulation de 500 véhicules envoyant simultanément leurs positions via gRPC.

— **Consultation Dashboard** : Requêtes GraphQL complexes simulant l'accès de 50 managers simultanés.

— **Stress Test Kafka** : Injection rapide d'alertes pour vérifier la capacité de traitement du `events-service`.

3.2. Résultats et Optimisations

Les tests ont mis en évidence la nécessité de configurer des *HPA (Horizontal Pod Autoscalers)* sur la Gateway et le service de localisation. Le passage à une communication asynchrone via Kafka pour les alertes a permis de maintenir une latence stable sous charge.

4. Déploiement Final avec Helm

Le déploiement a été uniformisé via **Helm** pour être « prod-ready ».

4.1. Organisation des Charts

— **`fleet-infra`** : Gère les dépendances (PostgreSQL, Kafka, Keycloak, Prometheus, Jaeger).

— **`fleet-app`** : Déploie l'ensemble des microservices et le frontend.

— **`fleet-observability`** : Configure Grafana et les dashboards.

4.2. Configuration Ingress et Sécurité

L'accès externe est géré par un Ingress Controller (Traefik ou Nginx) avec :

— Isolation des réseaux via des **Network Policies** Kubernetes.

— Gestion des secrets via des `Secret` Kubernetes (injectés au runtime).

— *Readiness* et *Liveness probes* configurées pour chaque service.

5. Documentation et Finalisation

5.1. Documentation Technique

La documentation a été centralisée et mise à jour :

— **README.md** : Guide d'installation rapide (Docker & Kubernetes).

— **API Docs** : Documentation OpenAPI pour les services REST et schéma GraphQL auto-généré.

— **ADR (Architecture Decision Records)** : Justification des choix technologiques (Kafka, Micro-frontend, gRPC).

5.2. Préparation de la Démonstration

Pour le rendu final, un scénario complet a été préparé :

1. Initialisation des données (Conducteurs, Véhicules).
2. Lancement d'une simulation GPS avec dépassement de vitesse.
3. Visualisation de l'alerte sur le frontend.
4. Consultation des traces dans Jaeger pour analyser le flux de l'alerte.

6. Livrables et État Final

Le système est désormais complet et déployable.

> **Note sur l'avancement :** Certains éléments de détail (polissage fin de certains dashboards, documentation exhaustive de chaque endpoint obscur) seront finalisés d'ici le rendu final, mais le cœur fonctionnel et l'infrastructure sont opérationnels et vérifiés par la CI/CD.

Livrable	Statut
Code source	Disponible sur GitHub (Monorepo)
Images Docker	Publiées sur GHCR
Charts Helm	Validés (Linting OK)
Rapports S1-S7	Termis et archivés
Documentation	Complète (README, ADR, PDF)

Table 1 – État des livrables pour le rendu final